

Count Arrays Formula

This problem looks complicated at first, but the counting pattern is actually very clean.

Key observation

An array of size n has exactly:

$$n - 1$$

adjacent pairs:

```
arr[0] - arr[1]
arr[1] - arr[2]
...
arr[n-2] - arr[n-1]
```

We need **exactly** k of these pairs to be equal.

Step 1: Choose which k adjacent positions will match

For example:

```
n = 5
k = 2
```

There are 4 adjacent relationships:

```
a0 =? a1
a1 =? a2
a2 =? a3
a3 =? a4
```

We need to choose exactly 2 of these 4 positions where the adjacent elements are equal.

Number of ways:

$$\binom{n-1}{k}$$

Step 2: Choose the first element

Each element can be from:

```
1 ... m
```

So:

$$m$$

choices for $arr[0]$.

Step 3: Fill the remaining elements

For every adjacent position:

If it is one of our chosen k matching positions

```
arr[i] = arr[i-1]
```

There is only:

$$1$$

choice.

If it is not a matching position

We need:

```
arr[i] != arr[i-1]
```

There are $m - 1$ choices.

Since there are:

$$(n - 1) - k$$

non-matching positions, we get:

$$(m - 1)^{n-1-k}$$

Final Formula

$$m \times \binom{n-1}{k} \times (m-1)^{n-1-k}$$

All operations are modulo:

```
1e9 + 7
```

Example 1

```
n = 3
m = 2
k = 1
```

Formula:

$$\begin{aligned} 2 \times \binom{2}{1} \times (2-1)^{2-1} \\ = 2 \times 2 \times 1 \\ = 4 \end{aligned}$$

✅ Answer = 4

Why choosing the positions works

Take:

```
n = 4
m = 3
k = 1
```

Suppose we choose this adjacent relationship to be equal:

```
a0 != a1 = a2 != a3
```

Now:

```
a0 → 3 choices
a1 → must differ from a0 → 2 choices
a2 → must equal a1 → 1 choice
a3 → must differ from a2 → 2 choices
```

Total:

$$3 \times 2 \times 1 \times 2 = 12$$

There are:

$$\binom{3}{1} = 3$$

ways to choose which adjacent pair matches.

Therefore:

$$3 \times 12 = 36$$

C++ Solution

We need to calculate:

$$\binom{n-1}{k}$$

efficiently because n can be up to 10^5 .

We use:

$$\binom{N}{R} = \frac{N!}{R!(N-R)!}$$

Under modulo, division is done using a **modular inverse**.

```
C++
class Solution {
public:
    static const int MOD = 1e9 + 7;

    long long power(long long a, long long b) {
        long long result = 1;

        while (b > 0) {
            if (b & 1) {
                result = result * a % MOD;
            }

            a = a * a % MOD;
            b >>= 1;
        }

        return result;
    }

    int countGoodArrays(int n, int m, int k) {
        // Precompute factorials
        vector<long long> fact(n);
        vector<long long> invFact(n);

        fact[0] = 1;

        for (int i = 1; i < n; i++) {
            fact[i] = fact[i - 1] * i % MOD;
        }

        // Modular inverse of factorial
        invFact[n - 1] = power(fact[n - 1], MOD - 2);

        for (int i = n - 2; i >= 0; i--) {
            invFact[i] = invFact[i + 1] * (i + 1) % MOD;
        }

        // Calculate C(n - 1, k)
        long long combinations =
            fact[n - 1] *
            invFact[k] % MOD *
            invFact[n - 1 - k] % MOD;

        // m * C(n-1, k) * (m-1)^(n-1-k)
        long long answer = m;

        answer = answer * combinations % MOD;
        answer = answer *
            power(m - 1, n - 1 - k) % MOD;

        return answer;
    }
};
```

Complexity

Time: $O(n + \log \text{MOD})$
 Space: $O(n)$

The main pattern to recognize

This is a **Combinatorics + Modular Exponentiation** problem:

Choose equal positions $\times$ choose first value $\times$ choose values at unequal positions

So the entire problem reduces to:

$$m \cdot \binom{n-1}{k} \cdot (m-1)^{n-k-1}$$

The important insight is that once you decide **which adjacent pairs are equal**, every remaining adjacent pair simply has $m - 1$ choices.

Yes. Let's forget the formula for a moment and understand **why we are multiplying these things**.

We will use a small example:

```
n = 4
m = 3
k = 1
```

This means:

- Array length = 4
- Every number can be 1, 2, 3
- We want **exactly 1 adjacent pair to be equal**

So our array is:

```
[a0, a1, a2, a3]
```

There are $n - 1 = 3$ adjacent comparisons:

```
a0 == a1 ?
a1 == a2 ?
a2 == a3 ?
```

We want **exactly one** of these to be true.

Step 1: Understand what we are choosing

There are 3 places where equality can happen:

```
Position 1: a0 == a1
Position 2: a1 == a2
Position 3: a2 == a3
```

Since $k = 1$, we need to choose **exactly one** of these positions.

For example, suppose we choose:

```
a1 == a2
```

Then the other two must NOT be equal.

So the array must follow this pattern:

```
a0 != a1 == a2 != a3
```

Notice that we are **not choosing the actual numbers yet**.

We are first deciding the **pattern**.

Step 2: How many patterns can we have?

We have 3 adjacent positions:

```
a0-a1
a1-a2
a2-a3
```

We need to choose 1 to be equal.

Possible choices:

```
1. a0 == a1
   a0 == a1 != a2 != a3

2. a1 == a2
   a0 != a1 == a2 != a3

3. a2 == a3
   a0 != a1 != a2 == a3
```

So there are:

```
3 ways
```

This is:

$C(3, 1)$

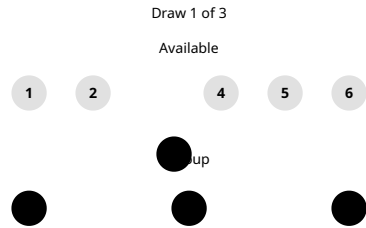
or generally:

$C(n - 1, k)$

Because:

```
n - 1 = total adjacent pairs
k      = pairs that should be equal
```

For combinations, the explorable idea is:



$\binom{6}{3} = \frac{6!}{3!(6-3)!} = 20$

After all 3 draws finish, their numerical order shows one combination; changing draw order does not create a new one.

Options Draws

[Give feedback](#)

Step 3: Now fix ONE pattern

Let's take this pattern:

```
a0 != a1 == a2 != a3
```

Now we will fill the array one element at a time.

Choose a0

Possible values:

- 1
- 2
- 3

So:

```
a0 → 3 choices
```

Because $m = 3$.

Choose a1

Our pattern says:

```
a0 != a1
```

Suppose:

```
a0 = 2
```

Then **a1** can be:

- 1
- 3

It cannot be 2.

So:

```
a1 → m - 1 choices
    → 2 choices
```

Choose a2

Our pattern says:

```
a1 == a2
```

Suppose:

```
a1 = 3
```

Then a2 **must be 3**.

```
a2 → only 1 choice
```

Because equality forces the value.

Choose a3

Our pattern says:

```
a2 != a3
```

Suppose:

```
a2 = 3
```

Then a3 can be:

```
1 or 2
```

So:

```
a3 → m - 1 choices
    → 2 choices
```

Put everything together

For this **one fixed pattern**:

```
a0 != a1 == a2 != a3
```

Choices are:

```
a0 → 3 choices
a1 → 2 choices
a2 → 1 choice
a3 → 2 choices
```

Therefore:

```
3 × 2 × 1 × 2 = 12
```

So this one equality pattern produces:

```
12 arrays
```

But we have 3 possible patterns

Remember:

```
1. == != !=
2. != == !=
3. != != ==
```

Each pattern has 12 possible arrays.

Therefore:

```
3 patterns × 12 arrays
= 36
```

So the answer is:

36

Now let's generalize

For an array of size n :

```
[a0, a1, a2, ..., a(n-1)]
```

There are:

```
n - 1
```

adjacent relationships.

For example:

```
a0 ? a1
a1 ? a2
a2 ? a3
...
```

We need exactly k of them to be:

```
==
```

So first:

Choose the k equal positions

From $n - 1$ positions:

```
C(n - 1, k)
```

ways.

Choose the first element

```
a0
```

can be anything from 1 to m .

Therefore:

```
m choices
```

For every remaining element

There are two cases.

Case 1: This position should be equal

```
previous == current
```

Example:

```
a1 = 5
a2 must be 5
```

Only:

```
1 choice
```

We have exactly k such positions.

So they contribute:

```
1 × 1 × 1 ...
```

which doesn't change anything.

Case 2: This position should NOT be equal

```
previous != current
```

If:

```
previous = 5
```

Then the current value can be any number except 5.

Out of m numbers:

```
m - 1 choices
```

How many such positions exist?

Total adjacent positions:

```
n - 1
```

Equal positions:

```
k
```

So non-equal positions:

```
(n - 1) - k
```

or:

```
n - 1 - k
```

Each has $m - 1$ choices:

```
(m - 1) × (m - 1) × ...
```

repeated $n - 1 - k$ times.

That becomes:

$$(m - 1)^{n-1-k}$$

The complete logic visually

```

      Array length = n
      a0      a1      a2      a3      ...
      ↓       ↓       ↓       ↓
m choices

      Each adjacent connection:
      == → 1 choice
      != → m - 1 choices

Choose which k connections are ==
      C(n-1, k)

Number of != connections:
      n - 1 - k

Final:
      m
      × C(n-1, k)
      × (m-1)^(n-1-k)
  
```

The main thing to understand is:

First choose WHERE the equal adjacent pairs will occur. Then, after fixing that pattern, count how many actual arrays can follow that pattern.

For example:

Pattern:

```
!= == !=
```


means:

```
a0 != a1 == a2 != a3
```

The pattern decides whether each new number has:

```
== → 1 choice
!= → m - 1 choices
```

That is the entire reason the formula works.

ChatGPT can make mistakes. Check important info.